

GUIDE COMPLET DU PROJET



TrafficGuard

Tout le projet expliqué en détail — et partagé en trois

Comprendre le sujet, les technologies et le code · lancer sur Google Colab · répondre à toutes les questions du jury.

Document d'étude : à lire et à maîtriser avant la soutenance. Aucune connaissance technique préalable n'est nécessaire.

Projet de Fin d'Études — Licence Génie Informatique — 2025-2026

Plan du guide

Ce guide est volontairement détaillé : c'est votre document d'étude. Il explique tout le projet pour que vous puissiez répondre à n'importe quelle question, sans avoir à relire le code le jour J.

Partie commune (toute l'équipe)

- Le projet en une page
- Le contexte, l'enjeu et la problématique
- Lancer le projet sur Google Colab (pas à pas)

Partie 1 — Présentateur 1 : Vue d'ensemble & côté web

- Architecture en 3 couches · interface · serveur Flask · temps réel · déploiement

Partie 2 — Présentateur 2 : L'intelligence artificielle (détection & suivi)

- Vision par ordinateur · YOLOv8 · COCO · ByteTrack · la classe Vehicule · la boucle d'analyse

Partie 3 — Présentateur 3 : La décision (juger l'arrêt)

- Zone de contrôle · mesure de l'arrêt · verdict · fiabilisation · résultats · limites

Partie commune : jour de la soutenance · problèmes courants · glossaire

À retenir : chacun apprend SA partie à fond, mais lit aussi les deux autres pour avoir la vue d'ensemble.
Le jury peut interroger n'importe qui.

Le projet en une page

TrafficGuard est une application web d'analyse vidéo. On lui fournit une vidéo de circulation filmée par une caméra fixe, et elle indique, véhicule par véhicule, lesquels ont respecté le panneau Stop et lesquels ne l'ont pas respecté.

Le système enchaîne trois opérations : il repère le panneau Stop dans l'image ; il repère les véhicules et les suit d'une image à l'autre (chacun reçoit un numéro stable) ; puis, pour chaque véhicule, il mesure s'il s'est immobilisé à proximité du Stop. S'il s'est arrêté, le verdict est « respecté » (cadre vert) ; sinon, « infraction » (cadre rouge).

À retenir : détecter une voiture près d'un Stop ne suffit pas. Une infraction, c'est NE PAS s'arrêter — un comportement qui se mesure dans le temps, sur plusieurs images. C'est l'idée centrale et originale du projet.

Le projet est composé de quelques fichiers, chacun avec un rôle précis :

Fichier	Son rôle
detection.py	Le cerveau. Lit la vidéo image par image, détecte panneaux et véhicules, mesure les arrêts, rend les verdicts.
app.py	Le pont (serveur Flask). Reçoit les demandes du navigateur et appelle detection.py.
templates/index.html	La structure de la page web : zones, boutons, compteurs.
static/style.css	L'apparence : thème sombre, couleurs, disposition, animations.
static/script.js	Le comportement de la page : envoi du fichier, mise à jour de l'affichage.
requirements.txt	La liste des bibliothèques à installer.
Dockerfile	La recette pour mettre l'application en ligne (déploiement).
yolov8n.pt	Le modèle d'intelligence artificielle pré-entraîné.

Le contexte, l'enjeu et la problématique

La sécurité routière et le panneau Stop

La sécurité routière est un enjeu majeur. Une grande partie des accidents survient aux intersections, souvent parce qu'un véhicule n'a pas respecté une règle de priorité. Le panneau Stop impose une règle simple mais stricte : tout véhicule doit marquer un arrêt complet avant de franchir l'intersection, afin de céder le passage et de vérifier que la voie est libre. Ne pas s'arrêter est une cause fréquente de collisions.

Les limites de la surveillance actuelle

Aujourd'hui, faire respecter cette règle repose surtout sur la présence d'agents. Cette approche a plusieurs limites : elle coûte cher (surveiller un seul carrefour en continu mobilise du personnel) ; elle ne peut pas être généralisée à de nombreuses intersections ; elle dépend de l'attention humaine, qui baisse avec la fatigue ; et elle ne produit aucune donnée exploitable pour analyser la situation.

L'intelligence artificielle et la vision par ordinateur offrent une alternative : analyser automatiquement des vidéos pour repérer les comportements infractionnels. Les modèles de détection d'objets sont aujourd'hui assez précis et rapides pour envisager une surveillance automatisée.

La problématique et l'idée centrale

La difficulté technique centrale du projet est la suivante : détecter une infraction au Stop ne se résume pas à constater qu'un véhicule franchit l'intersection. L'infraction, c'est précisément NE PAS s'arrêter. Or l'arrêt est un phénomène qui se déroule dans le temps : pour le constater, il faut observer le mouvement d'un véhicule sur plusieurs images, et non sur une seule.

La problématique se formule ainsi : comment détecter, de manière automatique et fiable, les véhicules qui ne respectent pas l'arrêt obligatoire à un panneau Stop, en s'appuyant sur l'analyse vidéo par intelligence artificielle ?

Les objectifs du projet

- Détecter les panneaux Stop dans une vidéo de circulation.
- Détecter et suivre les véhicules image par image, avec un identifiant stable.
- Mesurer si un véhicule marque un arrêt à proximité du Stop.
- Signaler les véhicules en infraction (absence d'arrêt).
- Comptabiliser distinctement infractions et arrêts respectés, et tenir un historique.
- Offrir une interface web moderne et accessible, sans installation.

Les technologies utilisées (vue d'ensemble)

Le projet combine plusieurs outils, tous libres et largement utilisés dans l'industrie. En voici la liste, avec ce que chacun apporte ; chaque partie du guide les approfondit ensuite.

- **Python** — le langage principal du projet. C'est le standard de l'intelligence artificielle, grâce à sa syntaxe claire et à ses nombreuses bibliothèques.
- **YOLOv8 (Ultralytics)** — le modèle qui détecte les panneaux et les véhicules en temps réel.
- **ByteTrack** — l'algorithme qui suit chaque véhicule en lui gardant un numéro stable.
- **OpenCV** — la bibliothèque de vision : lecture des vidéos, traitement et annotation des images.
- **NumPy** — les calculs numériques rapides (distances, mouvements).
- **Flask (et Unicorn)** — le serveur web qui relie l'IA au navigateur.
- **HTML, CSS, JavaScript** — l'interface web (structure, apparence, comportement).
- **Docker** — l'emballage du projet pour le déployer facilement en ligne.
- **Google Colab et ngrok** — pour exécuter le projet dans le navigateur (avec carte graphique) et obtenir un lien public.

Lancer le projet sur Google Colab (pas à pas)

Google Colab est un site web gratuit de Google qui exécute du code dans votre navigateur, sans rien installer sur votre ordinateur. Suivez ces étapes dans l'ordre ; chacune indique la commande à copier et ce que vous devez voir.

Étape 1 — Ouvrir Colab et créer un notebook

- Allez sur colab.research.google.com et connectez-vous avec un compte Google.
- Cliquez « Nouveau notebook ».

Ce que vous verrez : une « cellule » vide (un rectangle où l'on tape du code), avec un bouton ► à gauche.

Étape 2 — Activer la carte graphique (GPU)

- Menu « Exécution » > « Modifier le type d'exécution ».
- Choisissez « T4 GPU », puis « Enregistrer ».

Le GPU accélère énormément l'analyse. Sans lui, le projet fonctionne mais est plus lent.

Étape 3 — Importer le projet

À gauche, cliquez l'icône en forme de dossier, puis faites glisser le fichier `trafficguard_corrige.zip` dedans. Attendez la fin du téléversement.

Étape 4 — Décompresser le projet

Dans la cellule, collez et lancez (►) :

```
!unzip -q trafficguard_corrige.zip
%cd trafficguard
```

Ce que vous verrez : des lignes défiler, puis l'invite revenir. Le dossier est maintenant ouvert.

Étape 5 — Installer les bibliothèques

Ajoutez une cellule (« + Code »), puis :

```
!pip install -r requirements.txt
```

Ce que vous verrez : beaucoup de texte défiler pendant une à deux minutes. C'est normal.

Étape 6 — La clé ngrok

ngrok crée une adresse internet temporaire pour ouvrir l'application. Créez un compte gratuit sur dashboard.ngrok.com, ouvrez « Your Authtoken », copiez la clé, puis :

```
!pip install pyngrok
from pyngrok import ngrok
ngrok.set_auth_token("COLLEZ_VOTRE_CLE_ICI")
```

Étape 7 — Lancer l'application

```
from pyngrok import ngrok
import subprocess
tunnel = ngrok.connect(7860)
print("Ouvrez ce lien :", tunnel.public_url)
subprocess.run(["python", "app.py"])
```

Ce que vous verrez : une ligne « Ouvrez ce lien : <https://xxxx.ngrok-free.app> ». Cliquez dessus (puis « Visit Site » si une page d'avertissement apparaît).

À retenir : ne fermez jamais la cellule qui tourne pendant la démonstration : c'est elle qui fait fonctionner l'application.

PARTIE 1 · PRÉSENTATEUR 1

Vue d'ensemble & côté web : l'architecture, l'interface et le serveur

1.1 L'architecture en trois couches

Le projet est organisé en trois « couches » qui communiquent entre elles. C'est une bonne pratique de génie logiciel : on sépare les responsabilités pour rendre le système clair, robuste et facile à faire évoluer.

- La couche présentation (le navigateur) : tout ce que l'utilisateur voit et manipule. Fichiers : index.html, style.css, script.js.
- La couche applicative (le serveur Flask) : le pont entre le navigateur et le moteur. Fichier : app.py.
- La couche traitement (le moteur d'IA) : l'analyse réelle de la vidéo. Fichier : detection.py.

Le cycle complet : l'utilisateur clique « Lancer » dans le navigateur ; le navigateur envoie la demande au serveur ; le serveur démarre le moteur ; le moteur analyse la vidéo et prépare une image annotée et des compteurs ; le serveur les renvoie au navigateur ; le navigateur les affiche. Ce cycle se répète plusieurs fois par seconde pendant toute l'analyse.

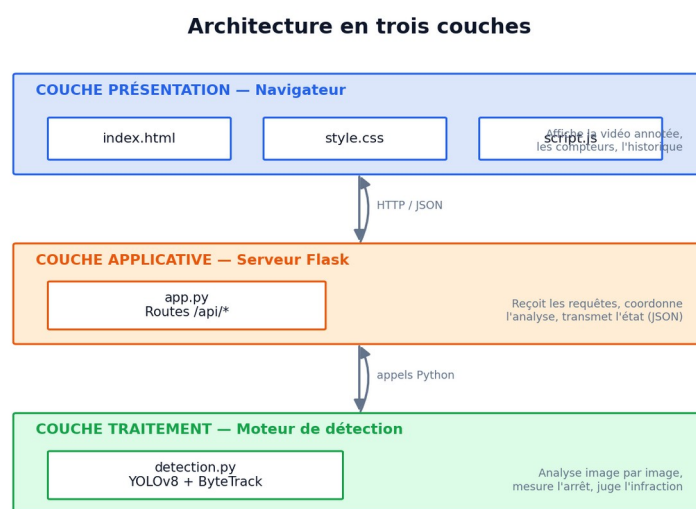


Figure 1 — L'architecture en trois couches de TrafficGuard

1.2 L'interface : index.html, style.css, script.js

L'interface se répartit en trois fichiers, comme pour construire une maison : un squelette, une décoration, et un comportement.

index.html — la structure

Ce fichier place les éléments de la page : la zone d'affichage de la vidéo annotée, les boutons (Vidéo/Image, Lancer, Pause, Arrêter), la zone de dépôt du fichier, le panneau latéral avec l'état du Stop, les compteurs et l'historique. C'est le squelette : il dit QUOI afficher et OÙ.

style.css — l'apparence

Ce fichier définit l'aspect visuel : le thème sombre, les couleurs (rouge pour les infractions, vert pour les arrêts respectés, bleu pour les véhicules détectés), la disposition des zones, les arrondis et les animations. Il rend l'interface agréable et lisible.

script.js — le comportement

Ce fichier rend la page vivante. Il réagit aux clics, envoie le fichier choisi au serveur, lance l'analyse, et surtout interroge le serveur en boucle pour rafraîchir l'écran. Concrètement, toutes les 250 millisecondes (soit environ quatre fois par seconde), il demande au serveur la dernière image annotée et les derniers compteurs, puis met à jour la page. Cette technique d'interrogation répétée s'appelle le « polling » : c'est elle qui donne l'impression d'un direct.

1.3 Le serveur Flask (app.py)

Flask est un micro-framework : un petit outil qui permet de transformer du code Python en application web. Dans app.py, on définit des « routes » : ce sont des adresses que le navigateur appelle, et chacune déclenche une action précise du côté serveur.

Route	Ce qu'elle fait
/	Renvoie la page web principale (l'interface).
/api/upload	Reçoit le fichier (vidéo ou image) envoyé par l'utilisateur et le sauvegarde.
/api/demarrer	Lance l'analyse d'une vidéo (dans un fil d'exécution séparé).
/api/analyser-image	Analyse une image fixe et renvoie directement le résultat.
/api/etat	Renvoie l'état courant (compteurs + image annotée). C'est elle que le navigateur appelle en boucle.
/api/arreter	Arrête l'analyse en cours.
/api/pause	Met en pause ou reprend l'analyse vidéo.

Détails importants : app.py crée UNE seule fois le moteur de détection au démarrage (le modèle d'IA est lourd à charger, on ne le fait donc qu'une fois). Il vérifie le type de fichier (vidéo ou image) et refuse les formats non supportés. Il refuse aussi de lancer deux analyses simultanées. Enfin, il ajoute des en-têtes (CORS) qui permettent à l'application de fonctionner correctement quand elle est exposée via ngrok.

1.4 Pourquoi l'écran ne se fige pas : thread et verrous

Analyser une vidéo peut durer plusieurs dizaines de secondes. Si le serveur faisait tout d'un seul bloc, l'écran resterait figé jusqu'à la fin. La solution est de lancer l'analyse dans un « fil d'exécution » séparé (un thread), qui travaille en arrière-plan. Pendant ce temps, le serveur reste disponible pour répondre aux demandes d'actualisation du navigateur.

Comme deux fils tournent en même temps (l'analyse et les réponses au navigateur), on utilise des « verrous » (locks) : ils garantissent que les deux fils n'accèdent jamais aux mêmes données au même instant, ce qui éviterait des erreurs ou des affichages incohérents.

1.5 Les deux modes et le déploiement

L'application a deux modes. Le mode vidéo réalise l'analyse complète, avec le jugement des infractions. Le mode image se contente de détecter les panneaux et les véhicules sur une photo, sans rendre de verdict — car une infraction se mesure dans le temps, ce qu'une image fixe ne permet pas.

Pour le déploiement, le projet fournit un Dockerfile : une recette qui décrit, étape par étape, comment installer l'application sur un serveur vierge. Cela permet de la mettre en ligne (par exemple sur Hugging Face Spaces). Pour les tests et la démonstration, on utilise Google Colab (pour le calcul) et ngrok (pour obtenir un lien public).

1.6 La modélisation : cas d'utilisation et séquence

Avant de programmer, nous avons modélisé le système avec le langage UML, qui sert à représenter un logiciel par des schémas. Deux schémas concernent surtout cette partie.

Le diagramme de cas d'utilisation (figure 2) montre ce que l'utilisateur peut faire avec le système : charger un média, lancer ou arrêter l'analyse, consulter les compteurs et l'historique. Les traitements internes (détecter, suivre, mesurer, juger) sont déclenchés automatiquement par le lancement de l'analyse.

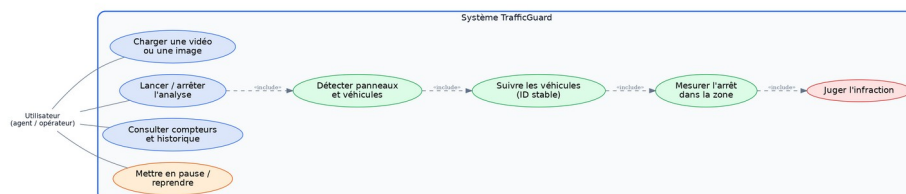


Figure 2 — Le diagramme de cas d'utilisation

Le diagramme de séquence (figure 3) montre le déroulement de l'analyse dans le temps : qui parle à qui, et dans quel ordre. On y voit le navigateur envoyer la vidéo, le serveur lancer le moteur dans un thread, puis le navigateur réclamer l'état toutes les 250 ms pour rafraîchir l'écran.

Diagramme de séquence — Analyse d'une vidéo

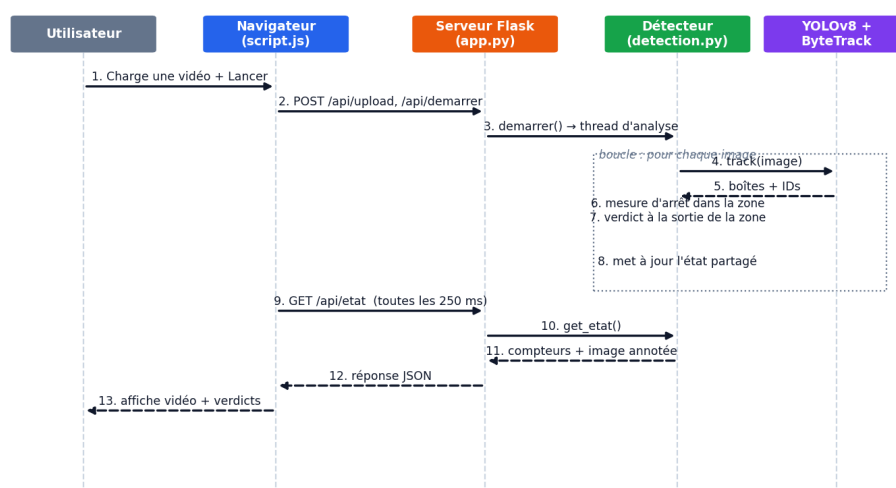


Figure 3 — Le diagramme de séquence de l'analyse

L'essentiel à retenir

- Trois couches : navigateur (affichage), Flask (serveur/pont), detection.py (moteur).
- index.html = structure, style.css = apparence, script.js = comportement (polling 250 ms).
- Flask expose des routes ; /api/etat est appelée en boucle pour rafraîchir l'écran.
- L'analyse tourne dans un thread séparé pour ne pas figer l'interface.

Questions du jury — Partie 1

Pourquoi une application web plutôt qu'un logiciel à installer ?

Parce qu'une application web s'utilise depuis n'importe quel navigateur, sans installation. On ouvre un lien et ça fonctionne, ce qui la rend accessible à tous.

Qu'est-ce que Flask et quel est son rôle ici ?

Flask est un petit framework qui permet de créer un site web en Python. Ici il sert de pont entre l'interface (le navigateur) et le moteur d'analyse (Python) : il reçoit les demandes et transmet les résultats.

Qu'est-ce qu'une route ?

Une adresse de l'application qui déclenche une action précise. Par exemple, /api/demarrer lance l'analyse, et /api/etat renvoie l'état courant.

Comment l'affichage se met-il à jour en temps réel ?

Le navigateur demande au serveur la dernière image et les compteurs environ quatre fois par seconde (le polling), et rafraîchit la page à chaque réponse.

Pourquoi l'analyse tourne-t-elle dans un thread séparé ?

Pour ne pas bloquer le serveur : pendant que l'analyse, longue, se déroule en arrière-plan, le serveur reste libre pour répondre au navigateur et l'écran continue de se mettre à jour.

Pourquoi parler de trois couches ?

Pour séparer clairement les responsabilités (affichage, serveur, calcul). Cela rend le code plus lisible, plus facile à corriger et à faire évoluer.

Que se passe-t-il si l'utilisateur lance deux analyses en même temps ?

Le système refuse : il vérifie qu'aucune analyse n'est déjà en cours avant d'en démarrer une autre, pour éviter les conflits.

À quoi servent les en-têtes CORS dans app.py ?

Ils autorisent l'application à fonctionner correctement lorsqu'elle est ouverte via une adresse externe comme celle de ngrok.

Pourquoi le modèle d'IA n'est-il chargé qu'une seule fois ?

Parce qu'il est lourd à charger. On le charge une fois au démarrage, puis on le réutilise pour toutes les images : c'est beaucoup plus rapide.

Que renvoie le serveur au navigateur ?

Un petit paquet de données au format JSON contenant les compteurs, l'état du panneau, l'historique, et l'image annotée encodée.

PARTIE 2 · PRÉSENTATEUR 2

L'intelligence artificielle : comment le système détecte et suit les véhicules

2.1 La vision par ordinateur

La vision par ordinateur est le domaine de l'informatique qui apprend aux machines à « comprendre » des images. Une image, pour un ordinateur, n'est qu'un tableau de nombres (les pixels). L'enjeu est de transformer ces nombres en informations utiles : ici, savoir où sont les véhicules et le panneau Stop.

2.2 YOLOv8 : la détection d'objets

Pour repérer les objets, nous utilisons YOLOv8, un modèle d'intelligence artificielle spécialisé dans la détection d'objets. Son nom, You Only Look Once (« on ne regarde qu'une fois »), résume son principe : il analyse l'image entière en une seule passe, ce qui le rend très rapide — un atout essentiel pour traiter une vidéo.

Pour chaque objet repéré, YOLO fournit trois informations : une boîte englobante (un rectangle qui entoure l'objet et indique sa position), une classe (de quel type d'objet il s'agit) et un score de confiance (à quel point le modèle est sûr de lui, entre 0 et 1). Nous utilisons la version « nano » (le fichier yolov8n.pt), la plus légère de la famille : elle offre un bon compromis entre vitesse et précision et fonctionne même sans matériel puissant.

Un réglage important est le seuil de confiance, fixé à 0,35 dans le code : toute détection dont le score est inférieur est ignorée. L'augmenter donne moins de détections, mais plus fiables ; le diminuer en donne davantage, au risque de fausses détections.

2.3 Le jeu de données COCO et les classes utilisées

YOLOv8 a été entraîné, avant notre projet, sur un grand ensemble d'images annotées appelé COCO, qui contient 80 catégories d'objets courants. Le modèle connaît donc déjà les objets qui nous intéressent. Nous n'utilisons que cinq classes :

Numéro	Objet	Rôle dans TrafficGuard
2	Voiture	Suivi et jugement
3	Moto	Suivi et jugement
5	Bus	Suivi et jugement
7	Camion	Suivi et jugement
11	Panneau Stop	Sert à placer la zone de contrôle

À retenir : nous n'avons pas entraîné le modèle nous-mêmes : nous utilisons un modèle déjà entraîné. Le ré-entraîner sur des images de routes (par exemple marocaines) est une amélioration possible que vous pouvez citer.

2.4 Le suivi : ByteTrack et l'identifiant stable

Détecter ne suffit pas. Sur l'image suivante, le modèle redétecte les objets, mais comment savoir que la voiture de l'image 51 est la même que celle de l'image 50 ? C'est le rôle du suivi (tracking). Nous utilisons ByteTrack, un algorithme de suivi intégré à YOLOv8. Il associe les détections d'une image à l'autre et attribue à chaque véhicule un identifiant (un numéro) qui reste le même tant que le véhicule est visible.

Le suivi est l'élément clé du projet : c'est lui qui permet de reconstituer la trajectoire d'un véhicule, donc de mesurer plus tard s'il s'est arrêté. Sans suivi, on ne verrait que des détections isolées, sans pouvoir relier les positions dans le temps.

Détail du code : pour éviter qu'un numéro soit « recyclé » et attribué par erreur à un autre véhicule (ce qui fausserait les comptes), nous associons chaque numéro fourni par ByteTrack à un identifiant global, propre à notre système et vraiment stable. C'est ce mécanisme qui garantit qu'un même véhicule n'est jamais compté deux fois.

2.5 La mémoire d'un véhicule : la classe Vehicule

Dans `detection.py`, chaque véhicule suivi est représenté par un petit objet appelé `Vehicule`. Il sert de mémoire : il conserve la suite des positions du centre du véhicule (image après image), ainsi que la taille de sa boîte englobante à chaque instant. Cette mémoire est indispensable, car c'est en observant comment le centre se déplace au fil des images que l'on déterminera si le véhicule est à l'arrêt.

La classe `Vehicule` retient aussi des informations de décision : si le véhicule a marqué l'arrêt, s'il a déjà été jugé, et son verdict final. Ces drapeaux garantissent que chaque véhicule n'est jugé qu'une seule fois.

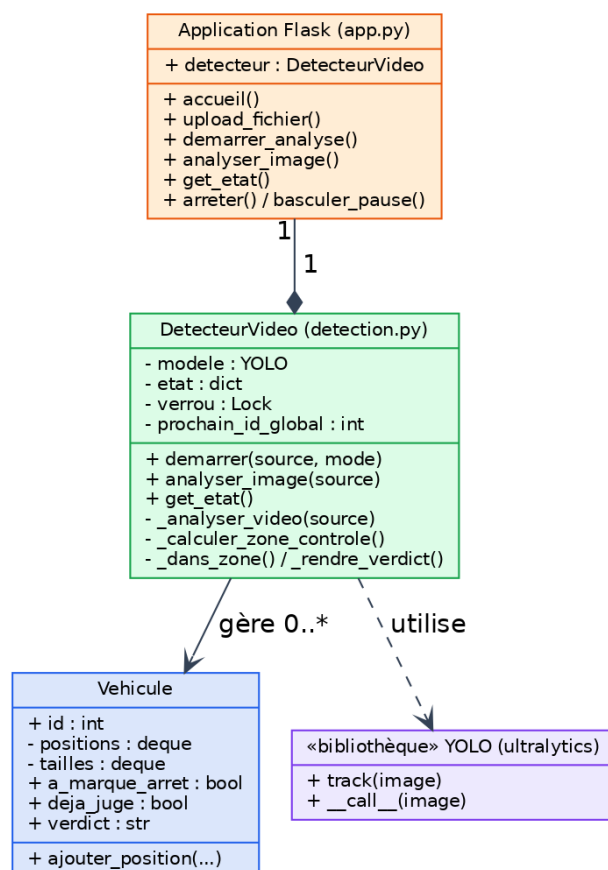


Figure 5 — Le diagramme de classes (*DetecteurVideo*, *Vehicule*)

2.6 La boucle d'analyse (vue d'ensemble du traitement)

Pour chaque image de la vidéo, le moteur effectue les opérations suivantes, dans l'ordre : il lit l'image avec OpenCV ; il la redimensionne si elle est trop grande (pour aller plus vite) ; il applique la détection et le suivi avec YOLOv8 et ByteTrack ; il repère les panneaux Stop ; il met à jour la mémoire de chaque véhicule ; il décide éventuellement d'un verdict ; il dessine les annotations (cadres, étiquettes, zone) ; enfin, il met à jour l'état partagé (compteurs et image), que le navigateur viendra lire.

OpenCV est la bibliothèque qui gère toute la partie image : lire la vidéo, redimensionner, pivoter si nécessaire, dessiner les rectangles et le texte, et encoder le résultat pour l'envoyer au navigateur. NumPy, de son côté, sert aux calculs rapides sur les nombres (par exemple la distance parcourue par un véhicule entre deux images).

Pipeline de traitement d'une vidéo



L'analyse s'exécute dans un thread séparé ; l'interface s'actualise toutes les 250 ms.

Figure 4 — Le pipeline de traitement, étape par étape

L'essentiel à retenir

- YOLOv8 détecte (boîte + classe + score) en une seule passe ; version nano, légère.
- Modèle pré-entraîné sur COCO ; classes utilisées : voiture, moto, bus, camion, panneau Stop.
- ByteTrack suit les véhicules avec un numéro stable ; un ID global évite le double comptage.
- La classe Vehicule mémorise les positions, indispensable pour mesurer l'arrêt.

Questions du jury — Partie 2

Quelle est la différence entre détecter et suivre ?

Détecter, c'est repérer un objet sur une image isolée. Suivre, c'est reconnaître que c'est le même véhicule d'une image à l'autre, en lui conservant le même numéro. Le suivi est indispensable pour mesurer un arrêt, car il faut relier les positions dans le temps.

Qu'est-ce que YOLO et pourquoi ce choix ?

YOLO est un modèle d'IA qui détecte des objets très rapidement, en analysant l'image entière en une seule passe. Nous utilisons la version nano, légère, qui fonctionne sans matériel spécialisé.

Que renvoie YOLO pour chaque objet ?

Une boîte (la position), une classe (le type d'objet) et un score de confiance (entre 0 et 1).

Qu'est-ce que ByteTrack ?

L'algorithme de suivi associé à YOLO : il garde un numéro stable pour chaque véhicule au fil des images.

Avez-vous entraîné le modèle vous-mêmes ?

Non, nous utilisons un modèle pré-entraîné sur le jeu COCO, qui connaît déjà les véhicules et les panneaux Stop. Le ré-entraîner sur des données routières est une perspective d'amélioration.

À quoi sert le seuil de confiance ?

C'est la certitude minimale exigée pour accepter une détection (0,35). En dessous, la détection est ignorée.

Comment évitez-vous de compter deux fois le même véhicule ?

Chaque véhicule garde un identifiant stable et n'est jugé qu'une seule fois ; un identifiant global évite que les numéros soient recyclés.

Qu'apportent OpenCV et NumPy ?

OpenCV gère les images (lecture, redimensionnement, dessin, encodage). NumPy effectue les calculs numériques rapides, comme les distances entre positions.

Pourquoi redimensionner les images avant l'analyse ?

Pour aller plus vite : une image plus petite se traite plus rapidement, sans perdre l'essentiel de l'information utile.

Le système gère-t-il plusieurs véhicules à la fois ?

Oui. Le suivi est multi-objets : chaque véhicule a son propre numéro et est analysé indépendamment des autres.

Que se passe-t-il si un véhicule est caché un instant (occlusion) ?

Son numéro est conservé un court moment grâce à une mémoire ; s'il réapparaît vite, il garde le même numéro. Sinon, il est oublié.

Pourquoi la version nano de YOLO et pas une plus grosse ?

Parce qu'elle est légère et rapide, et fonctionne sans matériel spécialisé. Une version plus grosse serait plus précise mais plus lente.

PARTIE 3 · PRÉSENTATEUR 3

La décision : comment le système juge l'arrêt (le cœur du projet)

3.1 La zone de contrôle

Autour du panneau Stop, le système définit une « zone de contrôle » : une région de l'image, proportionnelle à la taille du panneau détecté, où le respect de l'arrêt sera vérifié. Tant qu'un véhicule s'y trouve, on surveille son mouvement. Cela évite de juger des véhicules trop éloignés du panneau. À l'écran, cette zone est dessinée en orange. Si le panneau disparaît brièvement (par exemple masqué un instant), le système le mémorise pendant un court moment pour éviter que la zone ne « clignote ».

3.2 Mesurer l'arrêt (l'idée la plus importante)

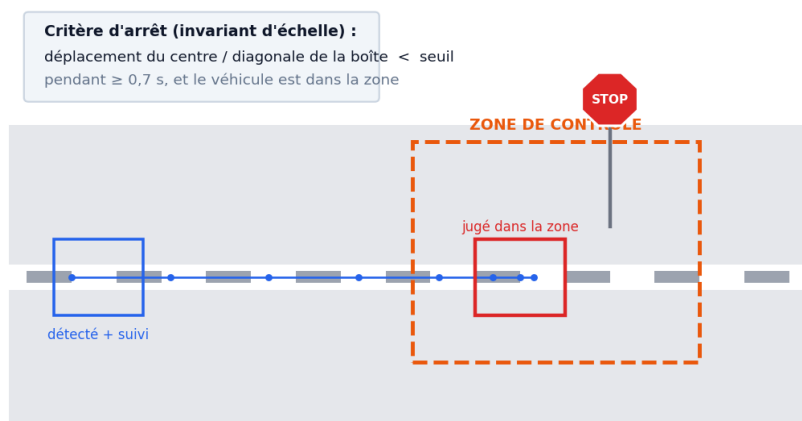
Pour savoir si un véhicule s'arrête, on observe de combien le centre de sa boîte se déplace entre deux images. Mais une difficulté apparaît : un véhicule loin de la caméra se déplace de peu de pixels même lorsqu'il roule, tandis qu'un véhicule proche se déplace de beaucoup de pixels. Comparer des pixels bruts serait donc injuste.

La solution : on divise le déplacement par la taille du véhicule (la diagonale de sa boîte). Ce « déplacement relatif » ne dépend plus de la distance à la caméra ni de la résolution : on dit qu'il est invariant à l'échelle. C'est une amélioration importante du projet.

Ensuite, pour ne pas se laisser tromper par les petits tremblements de la détection (la boîte bouge légèrement même quand le véhicule est immobile), on ne se fie pas à une seule image. On regarde la valeur médiane du déplacement relatif sur une courte durée (environ une demi-seconde). La médiane est robuste : un seul soubresaut ne la fait pas basculer. Si cette médiane reste très faible pendant que le véhicule est dans la zone, on conclut qu'il s'est arrêté.

À retenir : un véhicule est jugé « à l'arrêt » si, dans la zone du Stop, son centre bouge très peu par rapport à sa taille, pendant assez longtemps. Sinon, il est considéré comme ayant continué de rouler.

Zone de contrôle et mesure de l'arrêt



3.3 Le verdict : respecté ou infraction

Le système rend un verdict une seule fois par véhicule. Si le véhicule a marqué l'arrêt dans la zone, le verdict est « respecté » (cadre vert). S'il a traversé la zone sans s'arrêter, le verdict est « infraction » (cadre rouge), accompagné d'une bannière d'alerte.

Pour décider au bon moment, le code distingue deux situations. Un véhicule qui circule est jugé lorsqu'il dépasse le panneau (au moment où il s'en éloigne après l'avoir approché). Un véhicule à l'arrêt est jugé lorsqu'il quitte la zone. Conséquence importante : un véhicule simplement stationné sur le côté, hors de la trajectoire, n'est pas accusé à tort — c'est une garantie contre les fausses alertes.

3.4 Compteurs, historique, pause et mode image

- Les compteurs affichent en direct le nombre d'infractions, d'arrêts respectés et de véhicules suivis.
- L'historique présente les huit derniers événements, chacun horodaté et accompagné du numéro du véhicule.
- Le bouton Pause fige l'analyse sans la perdre ; Reprendre la relance — utile pour commenter un instant précis.
- Le mode image détecte panneaux et véhicules sur une photo, mais ne rend aucun verdict, car une infraction se mesure dans le temps. Cette limite est volontaire et assumée.

3.5 La fiabilisation : les améliorations apportées

Une première version du moteur fonctionnait, mais pouvait se tromper dans certains cas. Nous l'avons fiabilisée par plusieurs corrections. Ces améliorations constituent un apport d'ingénierie important du projet, à mettre en avant à la soutenance :

- Mesure de l'arrêt invariante à l'échelle : le déplacement est rapporté à la taille du véhicule, donc indépendant de la distance à la caméra.
- Mesure robuste au bruit : on utilise la médiane du mouvement sur une demi-seconde, et non une seule image, pour ne pas se faire piéger par les tremblements de la détection.
- L'arrêt n'est comptabilisé que dans la zone du Stop, pas n'importe où à l'écran.
- Un véhicule stationné hors trajectoire n'est plus accusé à tort, grâce à la distinction entre véhicule qui circule et véhicule à l'arrêt.
- Identifiants globaux stables, pour ne jamais compter deux fois le même véhicule.

3.6 Les réglages que l'on peut modifier

En tête du fichier `detection.py`, quelques réglages permettent d'adapter le système à différentes scènes. Les connaître permet de répondre à une question pointue du jury.

Réglage	Rôle	Valeur
SEUIL_CONFIANCE	Certitude minimale pour accepter une détection.	0,35
RATIO_ARRET	Mouvement maximal (rapporté à la taille) pour être jugé immobile.	0,008
DUREE_ARRET_MIN	Durée d'immobilité requise pour valider un arrêt.	0,5 s
MEMOIRE_VEHICULE	Durée de mémorisation d'un véhicule disparu (occlusion).	60 images
MEMOIRE_STOP	Mémorisation du panneau s'il disparaît brièvement.	90 images

3.7 Résultats, limites et perspectives

Sur la vidéo de test, le système se comporte conformément à son objectif : le panneau Stop est détecté de façon stable, les véhicules sont correctement suivis, et un verdict est rendu pour chaque véhicule concerné — il reconnaît un véhicule qui s'arrête (respecté) comme un véhicule qui ne s'arrête pas (infraction), sans accuser à tort un véhicule stationné hors trajectoire.

Le projet a des limites, qu'il faut présenter honnêtement : la mesure de l'arrêt suppose une caméra fixe ; la détection est moins fiable de nuit ou par mauvais temps ; une occlusion prolongée peut faire perdre le suivi d'un véhicule ; et le modèle générique n'est pas spécialisé pour le trafic. Ces limites tracent les perspectives : ré-entraîner le modèle sur des données routières, mener une évaluation chiffrée sur de nombreuses vidéos, ajouter la reconnaissance des plaques, générer des rapports automatiques, ou déployer sur un système embarqué.

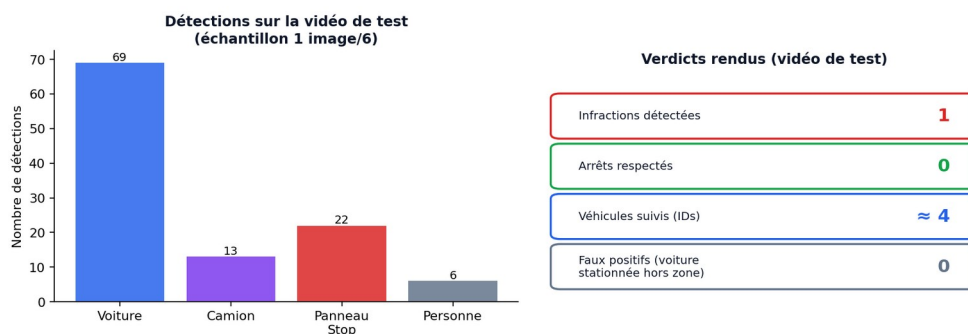


Figure 7 — Les résultats sur la vidéo de test

3.8 Deux exemples concrets, pas à pas

Pour bien comprendre, suivons deux véhicules.

Exemple A — une voiture qui s'arrête (respecté)

- La voiture est détectée et reçoit un numéro stable (par exemple #3).
- Elle entre dans la zone de contrôle, près du panneau (cadre bleu).

- Elle s'immobilise : son centre bouge très peu pendant plus d'une demi-seconde. Le système note « arrêt détecté ».
- Lorsqu'elle repart et quitte la zone, le verdict tombe : RESPECTÉ, cadre vert. Le compteur « respectées » augmente de 1.

Exemple B — une voiture qui grille le Stop (infraction)

- La voiture est détectée et suivie (par exemple #7).
- Elle traverse la zone de contrôle sans jamais s'immobiliser : son centre continue de bouger.
- Au moment où elle dépasse le panneau, le système constate qu'aucun arrêt n'a été mesuré.
- Le verdict tombe : INFRACTION, cadre rouge et bannière d'alerte. Le compteur « infractions » augmente de 1, et l'événement est horodaté dans l'historique.

L'essentiel à retenir

- La zone de contrôle limite le jugement aux abords du Stop.
- Arrêt = mouvement très faible (rapporté à la taille) pendant ~0,5 s, mesuré par la médiane (anti-bruit).
- Verdict une seule fois : vert (respecté) ou rouge (infraction).
- Un véhicule garé hors trajectoire n'est pas accusé à tort.

Questions du jury — Partie 3

Comment le système sait-il qu'un véhicule s'est arrêté ?

Il mesure de combien le centre du véhicule bouge entre deux images, rapporté à la taille du véhicule. Si ce mouvement reste très faible (en médiane, sur une demi-seconde) pendant qu'il est dans la zone du Stop, on considère qu'il s'est arrêté.

Pourquoi diviser le déplacement par la taille du véhicule ?

Pour être indépendant de la distance à la caméra : un véhicule loin bouge de peu de pixels, un véhicule proche de beaucoup. En rapportant à la taille, la mesure devient juste dans les deux cas (invariante à l'échelle).

Pourquoi utiliser la médiane et pas une seule image ?

Parce que la détection tremble légèrement : la boîte bouge un peu même à l'arrêt. La médiane sur une demi-seconde lisse ce bruit et évite les fausses conclusions.

Pourquoi le jugement est-il impossible sur une simple photo ?

Parce qu'une infraction, c'est l'absence d'arrêt — un comportement dans le temps. Une seule image ne montre aucun mouvement.

Un véhicule garé près du panneau est-il accusé à tort ?

Non. Le code distingue les véhicules qui circulent de ceux qui sont à l'arrêt hors trajectoire ; ces derniers ne sont pas jugés comme infraction.

Quelles sont les limites du système ?

Caméra fixe nécessaire ; fiabilité réduite de nuit ou par mauvais temps ; perte possible du suivi en cas d'occlusion ; modèle générique. Présenter ces limites fait partie d'une démarche scientifique honnête.

Quelle est sa précision et comment l'avez-vous testé ?

Nous l'avons validé sur une vidéo de test où il reconnaît correctement les arrêts et les infractions. Une évaluation chiffrée (précision, rappel) sur un grand nombre de vidéos annotées serait l'étape suivante.

En quoi est-ce différent d'un radar classique ?

Un radar mesure surtout la vitesse à l'aide de capteurs dédiés. Nous jugeons un comportement (l'arrêt) à partir d'une simple vidéo, sans capteur spécial, en analysant le mouvement dans le temps.

Que signifie « invariant à l'échelle » exactement ?

Cela veut dire que la mesure ne change pas selon que le véhicule est près ou loin de la caméra, car on rapporte son mouvement à sa propre taille.

Pourquoi un seul verdict par véhicule ?

Pour éviter qu'un même véhicule soit compté plusieurs fois. Un drapeau marque qu'il a déjà été jugé.

Comment régleriez-vous le système si une vidéo donne trop d'infractions ?

On peut ajuster les réglages : augmenter légèrement `RATIO_ARRET` ou réduire `DUREE_ARRET_MIN`, pour considérer l'arrêt un peu plus facilement.

Le système pourrait-il fonctionner en direct sur une vraie caméra ?

Oui, le principe le permet, à condition d'une caméra fixe et d'un ordinateur assez puissant, idéalement avec une carte graphique.

Annexe — Le code expliqué, fichier par fichier

Cette annexe parcourt les parties les plus importantes du code, avec de courts extraits et leur explication en français. Vous n'avez pas besoin de la connaître par cœur, mais la lire vous permettra de répondre avec assurance si le jury ouvre un fichier.

A.1 app.py — le serveur

Chaque « route » est une adresse de l'application. Par exemple, celle qui renvoie l'état courant :

```
@app.route("/api/etat")
def get_etat():
    return jsonify(detecteur.get_etat())
```

Traduction : quand le navigateur appelle l'adresse /api/etat, le serveur demande au moteur son état (compteurs, image annotée) et le renvoie au format JSON. C'est cette route que le navigateur interroge en boucle.

Le lancement d'une analyse se fait dans un thread, pour ne pas bloquer le serveur :

```
thread = threading.Thread(target=self._analyser_video,
                          args=(source,), daemon=True)
thread.start()
```

Traduction : on démarre la fonction d'analyse dans un fil d'exécution séparé, qui travaille en arrière-plan pendant que le serveur continue de répondre.

A.2 detection.py — les réglages

Tout en haut du fichier, des réglages clairs permettent de calibrer le système :

```
SEUIL_CONFIANCE = 0.35    # certitude minimale
RATIO_ARRET = 0.008      # mouvement max pour etre immobile
DUREE_ARRET_MIN = 0.5    # secondes d immobilite
CLASSES_VEHICULES = {2, 3, 5, 7} # voiture, moto, bus, camion
CLASSE_STOP = 11        # panneau Stop
```

Traduction : on regroupe les valeurs importantes au même endroit, ce qui rend le système facile à ajuster. Les numéros 2, 3, 5, 7 et 11 sont les catégories du jeu COCO que nous utilisons.

A.3 detection.py — la détection et le suivi en une ligne

Le cœur de l'IA tient en un appel : il fait à la fois la détection et le suivi.

```
resultats = self.modele.track(
    frame, conf=SEUIL_CONFIANCE,
    persist=True, tracker="bytetrack.yaml")
```

Traduction : pour l'image courante (frame), YOLO détecte les objets et ByteTrack leur attribue un numéro stable (persist=True signifie « garde la mémoire des images précédentes »).

A.4 detection.py — mesurer l'arrêt

Pour chaque véhicule, on calcule de combien son centre a bougé, rapporté à sa taille :

```
deplacement = distance(centre_actuel, centre_precedent)
deplacement_relatif = deplacement / taille_du_vehicule
# arret valide si la mediane sur ~0,5 s reste < RATIO_ARRET
```

Traduction : on ne compare pas des pixels bruts mais un mouvement rapporté à la taille (invariant à l'échelle), et on regarde la médiane sur une demi-seconde pour ignorer les petits tremblements. Si ce mouvement reste minuscule, le véhicule est « à l'arrêt ».

A.5 detection.py — rendre le verdict

Quand un véhicule a été vu dans la zone et qu'on peut conclure, on rend le verdict une seule fois :

```
if vehicule.a_marque_arret:
    verdict = "respecte" # cadre vert
else:
    verdict = "infraction" # cadre rouge
```

Traduction : s'il s'est arrêté, c'est « respecté » ; sinon, « infraction ». Le drapeau `deja_juge` garantit qu'on ne le compte qu'une fois.

A.6 script.js — la mise à jour de l'écran

Côté navigateur, on demande l'état au serveur quatre fois par seconde :

```
setInterval(actualiser, 250); // toutes les 250 ms
```

Traduction : la fonction `actualiser` est appelée automatiquement toutes les 250 millisecondes ; elle récupère la dernière image et les compteurs, et met l'écran à jour. C'est ce qui donne l'effet « temps réel ».

Le jour de la soutenance (pour tous)

Déroulé conseillé de la démonstration (5 à 7 minutes)

- Lancez l'application AVANT de passer (étapes Colab faites à l'avance).
- Présentez l'interface : la zone vidéo à gauche, les compteurs et l'historique à droite.
- Chargez la vidéo, lancez l'analyse, et commentez à voix haute : panneau détecté, véhicules suivis, mesure de l'arrêt.
- Au moment d'un verdict, soulignez : cadre vert = respecté, cadre rouge = infraction.
- Terminez sur les compteurs et l'historique horodaté.

Checklist à préparer à l'avance

- Application déjà lancée et testée 15 minutes avant.
- Vidéo de test prête, et une capture ou courte vidéo de secours au cas où la démo planterait.
- Connexion internet vérifiée (Colab et ngrok en ont besoin).
- Rapport et présentation ouverts ; rôle de chacun défini ; qui manipule l'ordinateur.

Conseil de prise de parole

Chacun présente sa partie avec ses propres mots, puis passe la parole (« Je laisse la parole à ... qui va vous présenter ... »). Restez calmes, parlez lentement, et n'ayez pas peur de dire « c'est une piste d'amélioration que nous avons identifiée » si une question dépasse le périmètre du projet.

Problèmes courants et solutions

Problème	Solution
Le lien ngrok ne s'ouvre pas	Vérifier la clé ngrok et que la cellule de lancement tourne toujours.
« Module not found »	Relancer : <code>!pip install -r requirements.txt</code>
Aucune détection à l'écran	Vidéo de mauvaise qualité ou panneau trop petit ; essayer une autre vidéo.
Trop d'alertes	L'arrêt est jugé trop sévèrement : on peut ajuster <code>RATIO_ARRET</code> ou <code>DUREE_ARRET_MIN</code> .
Analyse très lente	Vérifier que le GPU T4 est activé (Étape 2).
La page ne se met pas à jour	La cellule Colab s'est arrêtée : relancez le lancement.

Glossaire

- Détection : repérer un objet sur une image (boîte + classe + score).
- Suivi (tracking) : reconnaître le même objet d'une image à l'autre, avec un numéro stable.
- YOLOv8 : modèle d'IA de détection d'objets, rapide (une seule passe).
- ByteTrack : algorithme de suivi associé à YOLO.
- COCO : grand jeu d'images sur lequel le modèle a été entraîné.
- Boîte englobante : le rectangle qui entoure un objet détecté.
- Score de confiance : la certitude du modèle pour une détection (entre 0 et 1).
- Zone de contrôle : la région autour du Stop où l'on vérifie l'arrêt.
- Invariant à l'échelle : qui ne dépend pas de la distance à la caméra.
- Médiane : la valeur du milieu d'une série ; robuste aux valeurs extrêmes.
- Thread (fil d'exécution) : une tâche qui tourne en arrière-plan sans bloquer le reste.
- Polling : le navigateur interroge le serveur plusieurs fois par seconde pour rafraîchir l'affichage.
- Flask : micro-framework pour créer une application web en Python.
- Verdict : la décision du système pour un véhicule (respecté ou infraction).

Fiches mémo de soutenance

Une carte par présentateur, à relire juste avant de passer. L'essentiel à dire et à savoir, en quelques points.

Présentateur 1 — Vue d'ensemble & côté web

- Le projet : une appli web qui dit, pour chaque véhicule, s'il a respecté le Stop.
- Trois couches : navigateur (affichage) · serveur Flask (pont) · moteur detection.py (calcul).
- L'écran se met à jour 4 fois/seconde (polling) ; l'analyse tourne dans un thread pour ne pas figer l'interface.
- Phrase clé : « détecter ne suffit pas, on mesure le comportement ».

Présentateur 2 — Détection & suivi (l'IA)

- YOLOv8 détecte (boîte + classe + score) en une passe ; modèle pré-entraîné, version nano.
- ByteTrack suit chaque véhicule avec un numéro stable (pas de double comptage).
- Classes utilisées : voiture, moto, bus, camion, panneau Stop.
- La classe Vehicule mémorise les positions : indispensable pour mesurer l'arrêt.

Présentateur 3 — La décision (juger l'arrêt)

- Zone de contrôle autour du Stop ; on n'y juge que les véhicules proches.
- Arrêt = mouvement très faible (rapporté à la taille, donc invariant à l'échelle) pendant ~0,5 s.
- On utilise la médiane pour ignorer le bruit ; verdict une seule fois : vert ou rouge.
- Limites assumées : caméra fixe, nuit, occlusion, modèle générique.

Bonne soutenance. Chacun maîtrise sa partie, tout le monde connaît les grandes lignes des autres, et vous assumez les limites avec honnêteté : c'est exactement ce que le jury attend.